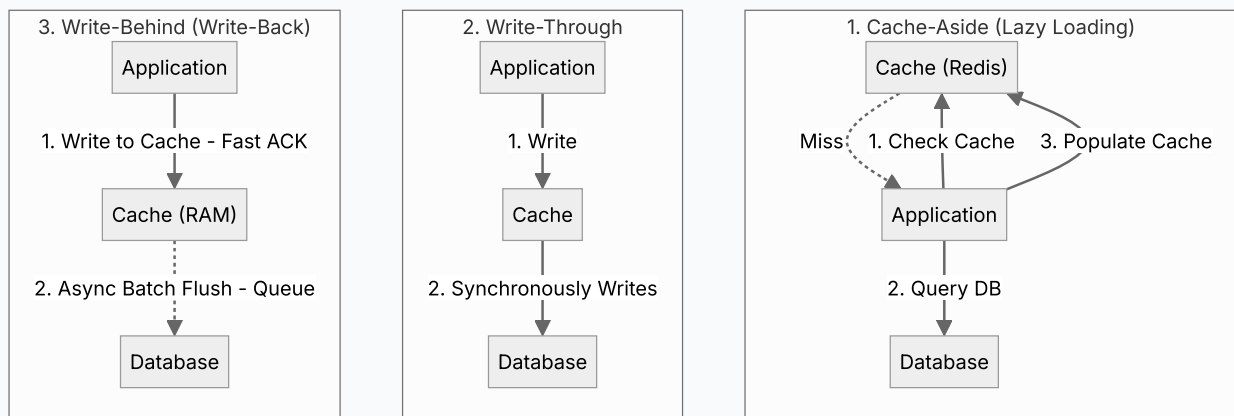


HLD Chapter 4: Caching, Load Balancing, CDNs & Edge Networks

Core Learning Objective: Master high-performance data delivery architectures. Understand the 4 write policies, how to defend against cache stampedes, L4 vs L7 load balancing, and global CDN edge caching.

1. Caching Strategies & Write Policies



Write Policies Comparison Matrix

Strategy	Read Latency	Write Latency	Data Consistency	Crash Risk
Cache-Aside	Fast on hit, slow on initial miss	Fast	Potential stale reads until TTL expires	Zero data loss (DB is source of truth)
Write-Through	Always fast (warm cache)	Higher (dual write: Cache + DB)	High consistency	Zero data loss
Write-Behind	Ultra fast	Minimal (RAM write speed)	Eventual consistency	High: Power loss before async flush loses data!

2. The 4 Fatal Cache Failure Modes & Defenses

1. Cache Stampede (Thundering Herd)

- **What happens:** A hot key (e.g. homepage top news viewed by 50,000 QPS) expires from cache. 50,000 concurrent threads simultaneously miss the cache and overwhelm the underlying SQL database with identical queries, causing a total database outage.
- **Defense: Distributed Mutex Lock / Probabilistic Early Expiration (XFetch):**



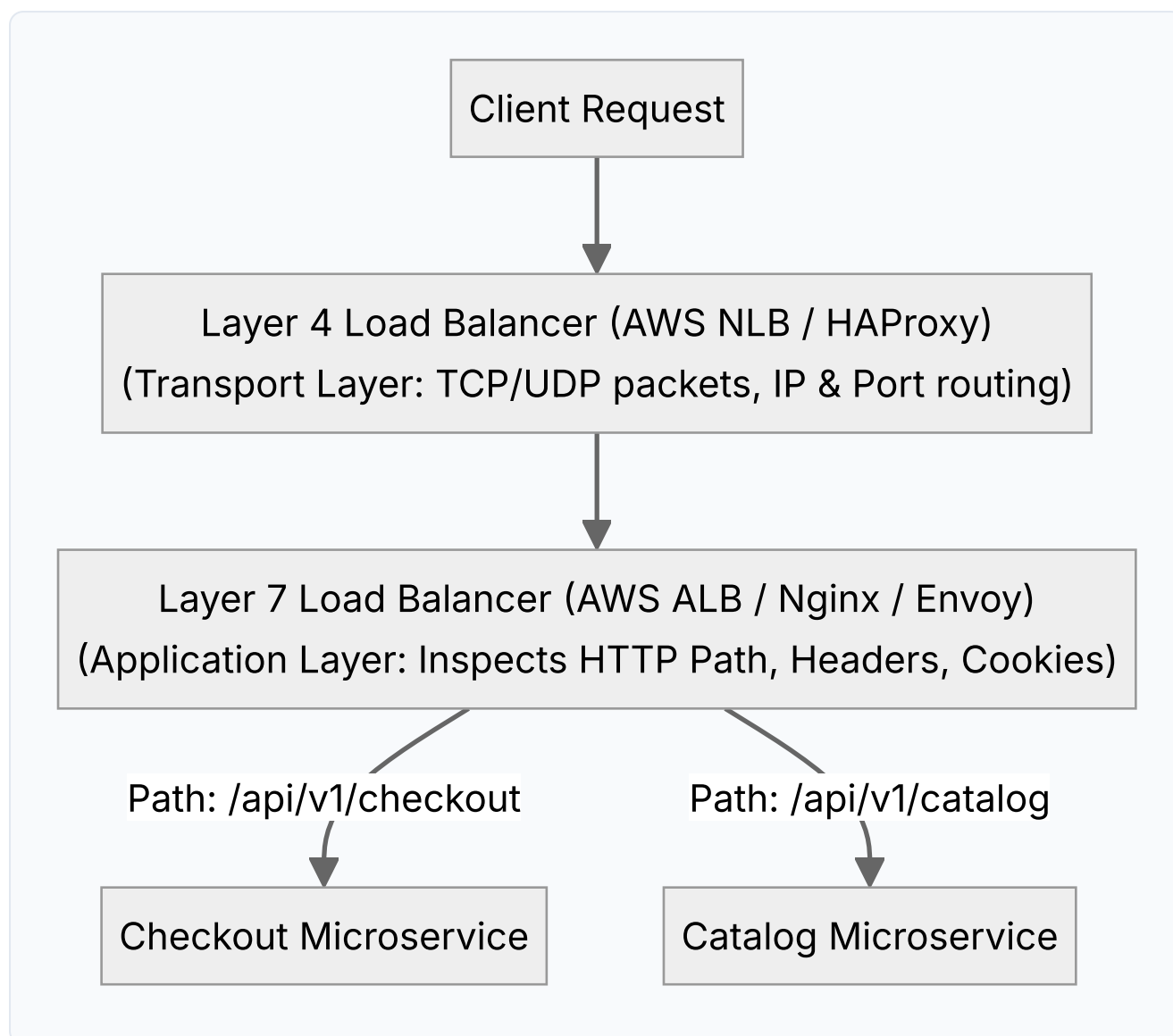
2. Cache Penetration

- **What happens:** An attacker queries non-existent keys (e.g., `user_id = -999999`). The cache misses, queries the DB, finds nothing, and does not cache the result. Every malicious request hits the database directly.
- **Defense: Bloom Filters** at the gateway (guarantees a key definitely does not exist in $O(1)$) OR cache `null` with a short 60-second TTL.

3. Cache Avalanche

- **What happens:** Hundreds of thousands of keys were populated at the same time with identical TTLs (e.g., exactly 24 hours). When the 24 hours expire, they all drop out simultaneously, crashing the DB.
- **Defense: Add Jitter to TTL:** `TTL = base_ttl + random.randint(0, 300)`.

3. Load Balancing: Layer 4 vs Layer 7



Dimension	Layer 4 (Transport)	Layer 7 (Application)
Data Inspected	IP addresses & TCP/UDP Ports	Full HTTP/HTTPS payload, URI path, headers, cookies
TLS / SSL Termination	Pass-through (Client negotiates directly)	Terminates SSL at the load balancer
Throughput & Latency	Millions of QPS; microsecond latency (zero packet inspection)	High latency overhead; smart routing decisions
Routing Capability	Round-robin across static IP pools	Path-based routing, A/B testing, auth inspection

4. Content Delivery Networks (CDNs) & Anycast Routing

A CDN is a geographically distributed network of **Edge Proxy Servers (Points of Presence - POPs)** that cache static and streaming content (images, videos, JS/CSS bundles) close to the user.

- **Anycast DNS Routing:** Multiple worldwide CDN POPs advertise the **exact same public IP address** via BGP (Border Gateway Protocol). The internet's routing fabric automatically directs user packets to the topologically closest datacenter.
- **Origin Shielding:** A secondary caching layer between CDN edge servers and your main datacenter to ensure origin servers are never flooded during cache misses.

5. Runnable Models: Stampede Protection, TTL Jitter and Negative Caching

Cache stampede: 50 threads, one expensive loader



Other stampede defences: **probabilistic early refresh** (refresh a hot key slightly before it expires, with some randomness), a **lock or lease per key** in a shared cache, and **serve stale while revalidating** (return the old value while one worker refreshes it).

Avalanche and penetration



Penetration (requests for keys that never exist) is blocked by **negative caching** with a short TTL, or by a Bloom filter in front of the cache that answers "definitely not present". Avalanche (many keys expiring together) is blocked by **TTL jitter** and by keeping the cache cluster highly available.

Which write policy for which workload

Workload	Policy	Why
Mostly reads, tolerates brief staleness	Cache-aside with TTL	Simple, the database stays the source of truth
Reads must see the latest write	Write-through	Cache and store are updated together; writes pay both latencies
Write bursts, loss of the last few seconds acceptable	Write-back	Fast writes; risk of loss if the cache dies before flushing
Rarely read after write	Write-around	Do not pollute the cache with data nobody reads

Further Reading

- [Redis documentation](#)
- [NGINX load balancing](#)
- [Wikipedia: Consistent hashing](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. Cache-aside versus write-through?

Cache-aside: the app loads on miss and writes to the DB then invalidates. Write-through: writes go through the cache to the store synchronously.

▼ 2. What is a cache stampede and a fix?

Many requests miss at once and hammer the origin. Use request coalescing, jittered TTLs or probabilistic early refresh.

▼ 3. Layer 4 versus layer 7 load balancing?

L4 balances on IP/port (fast, protocol-agnostic); L7 understands HTTP and can route on path, headers, cookies.

▼ 4. What is consistent hashing for?

Distributing keys so adding/removing a node remaps only about 1/N of them.